

Unsupervised Clustering of Optical-Fibre OTDR Signals

Week 3 — K-Means Clustering Assignment

Github [Code: week3-k-means-clustering](#)

Dataset: [Optical Fibre – Fault Detection \(Kaggle\)](#)

Notebook นี้วิเคราะห์รูปแบบสัญญาณ Optical Time-Domain Reflectometer (OTDR) แบบ **unsupervised learning** ด้วย K-Means โดยไม่ใช้ fault label ในการฝึกโมเดล

tl;dr

- หลังตัด exported row ID และ exact duplicates 6,824 แถว เหลือข้อมูล **119,008 แถว × 31 clustering features** โดยไม่มี missing หรือ infinite values.
- เกณฑ์หลัก Silhouette Score เลือก **K=2 (0.2420)**; Davies–Bouldin เลือก **K=3 (1.5017)** แทน จึงมีหลักฐานว่าคำตอบไม่ได้เด็ดขาดเพียงค่าเดียว.
- โมเดล K=2 แบ่งข้อมูลเป็น **74,436 แถว (62.5%)** และ **44,572 แถว (37.5%)**.
- PCA สองแกนอธิบาย variance รวม **60.0%** และแสดงสอง signal families กว้าง ๆ แต่ยังมีการซ้อนทับกัน.
- การเทียบ label ภายหลังให้ ARI **0.0973** และ NMI **0.1873** ; clusters จึงไม่ใช่ตัวแทนแบบหนึ่งต่อหนึ่งของ fault classes ทั้งแปด.

Context & Methods

Research question: Can K-Means discover meaningful groups of optical-fibre signal patterns from OTDR measurements without using predefined fault labels?

- **Clustering inputs:** SNR and the normalized OTDR sequence P1 – P30 .
- **Excluded from training:** exported row ID, Class , Position , Reflectance , and loss . The last four columns are retained only for post-hoc interpretation.
- **Preprocessing:** remove exact duplicate records, median imputation, then StandardScaler .
- **Models:** K-Means for $K = 2, \dots, 10$, n_init=20 , random_state=42 .
- **Selection:** prefer high Silhouette Score and low Davies–Bouldin Score; inertia is shown as a supporting elbow diagnostic.
- **Visualization:** PCA reduces the 31 standardized inputs to two dimensions only for display; K-Means is fitted in the full 31-dimensional space.

The source is the 2022 IEEE DataPort [Dataset for Optical fiber faults](#), mirrored on Kaggle. The class-name interpretation is also documented in [Titouni et al. \(2025\)](#), [IEEE OJ-COMS](#).

Key assumptions

1. Exact duplicate rows are repeated observations rather than independent optical measurements. Keeping thousands of copies would overweight a few signal shapes.
2. Euclidean distance on standardized variables is a useful first approximation for comparing these fixed-length OTDR traces.
3. `Class` is used only after clustering as an external interpretation check; it never enters imputation, scaling, PCA fitting, or K-Means fitting.
4. Silhouette Score is estimated on one fixed, reproducible 10,000-row sample because its pairwise-distance computation is quadratic in the number of observations.

Requirement 1: Choose any numeric dataset from Kaggle.

Selected dataset: [Optical Fibre – Fault Detection \(Kaggle\)](#), a numeric OTDR signal dataset in photonics and optical communication.

Data

```
In [1]: from __future__ import annotations

import json
import platform
import time
from pathlib import Path

import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import sklearn
from IPython.display import Markdown, display
from matplotlib.colors import LinearSegmentedColormap
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    adjusted_rand_score,
    davies_bouldin_score,
    normalized_mutual_info_score,
    silhouette_score,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

def find_project_root(start: Path) -> Path:
    for candidate in (start, *start.parents):
        if (candidate / "config.json").exists() and (candidate / "data").exists():
            return candidate
    raise FileNotFoundError(
        "Could not locate config.json and data/ from the current path."
    )

PROJECT_ROOT = find_project_root(Path.cwd().resolve())
```

```

CONFIG_PATH = PROJECT_ROOT / "config.json"
RAW_DATA_PATH = PROJECT_ROOT / "data" / "raw" / "OTDR_data.csv"
METADATA_PATH = PROJECT_ROOT / "data" / "raw" / "OTDR_data.metadata.json"
FIGURE_DIR = PROJECT_ROOT / "outputs" / "figures"
TABLE_DIR = PROJECT_ROOT / "outputs" / "tables"
FIGURE_DIR.mkdir(parents=True, exist_ok=True)
TABLE_DIR.mkdir(parents=True, exist_ok=True)

with CONFIG_PATH.open(encoding="utf-8") as file_handle:
    config = json.load(file_handle)
with METADATA_PATH.open(encoding="utf-8") as file_handle:
    source_metadata = json.load(file_handle)

analysis_config = config["analysis"]
feature_columns = analysis_config["feature_columns"]
held_out_columns = analysis_config["held_out_columns"]
k_values = analysis_config["k_values"]
random_state = analysis_config["random_state"]

sns.set_theme(context="notebook", style="whitegrid", font_scale=1.0)
plt.rcParams.update(
    {
        "figure.dpi": 120,
        "savefig.dpi": 180,
        "axes.titleweight": "bold",
        "axes.labelcolor": "#262626",
        "text.color": "#262626",
        "grid.color": "#D9D9D6",
        "grid.linewidth": 0.6,
    }
)
COLORS = {
    "blue": "#2F6B8A",
    "gold": "#C6922B",
    "orange": "#D9793D",
    "charcoal": "#333333",
    "grey": "#7A7A7A",
}

environment = pd.DataFrame(
    {
        "Component": ["Python", "pandas", "NumPy", "scikit-learn", "Matplotlib"],
        "Version": [
            platform.python_version(),
            pd.__version__,
            np.__version__,
            sklearn.__version__,
            mpl.__version__,
        ],
    }
)
display(environment)

```

	Component	Version
0	Python	3.12.14
1	pandas	3.0.6
2	NumPy	2.5.3
3	scikit-learn	1.9.1
4	Matplotlib	3.11.2

Requirement 2: Briefly describe the data (rows, columns, features).

Data at a glance: 125,832 rows × 36 numeric source columns. The clustering inputs are 31 signal features (SNR and P1 – P30); Class , Position , Reflectance , and loss are held out, and Unnamed: 0 is an exported row ID.

1. Load and verify the source data

```
In [2]: assert RAW_DATA_PATH.exists(), (
        "Run `download-otdr-data` before executing this notebook."
    )
assert RAW_DATA_PATH.stat().st_size == source_metadata["size_bytes"]
assert source_metadata["dataset_handle"] == config["dataset"]["handle"]

raw_data = pd.read_csv(RAW_DATA_PATH)
expected_columns = (
    ["Unnamed: 0"] + feature_columns + held_out_columns
)
assert raw_data.columns.tolist() == expected_columns

dataset_overview = pd.DataFrame(
    {
        "Measure": [
            "Rows",
            "Columns",
            "Numeric columns",
            "Non-numeric columns",
            "Source size (MiB)",
            "Kaggle version",
        ],
        "Value": [
            f"{len(raw_data):,}",
            raw_data.shape[1],
            raw_data.select_dtypes(include=np.number).shape[1],
            raw_data.select_dtypes(exclude=np.number).shape[1],
            f"{RAW_DATA_PATH.stat().st_size / 1024**2:,.2f}",
            source_metadata["dataset_version"],
        ],
    }
)
display(dataset_overview)
print(f"Source: {source_metadata['dataset_url']}")
print(f"SHA-256: {source_metadata['sha256']}")
```

	Measure	Value
0	Rows	125,832
1	Columns	36
2	Numeric columns	36
3	Non-numeric columns	0
4	Source size (MiB)	70.94
5	Kaggle version	1

Source: <https://www.kaggle.com/datasets/yogi2727/optical-fibre-fault-detection>

SHA-256: aa2fbf1fe327686483d6c1a99ceefcd87e11d152a9d869b899cfe2132900b71f

2. Preview representative fields

```
In [3]: preview_columns = [
        "SNR", "P1", "P5", "P10", "P15", "P20", "P25", "P30",
        "Class", "Position", "Reflectance", "loss",
    ]
display(raw_data[preview_columns].head(8).round(4))
```

	SNR	P1	P5	P10	P15	P20	P25	P30	Class	Position	Reflec
0	10.0936	0.9900	0.8384	0.8462	0.7716	0.4693	0.3456	0.2808	1.0	0.08	
1	0.2193	0.3412	0.6485	0.1934	0.4743	0.5658	0.3845	0.6793	3.0	0.11	
2	9.2824	0.4383	0.3599	0.1483	0.0000	0.6640	0.9101	0.7235	0.0	0.00	
3	8.8034	0.6749	0.4751	1.0000	0.8158	0.7930	0.4112	0.0000	1.0	0.09	
4	16.4855	0.7671	0.0550	0.9463	0.5235	0.4965	0.4578	0.0282	0.0	0.00	
5	0.6184	0.5977	0.5489	0.5805	0.3028	0.6472	0.8664	0.0650	5.0	0.09	
6	23.0000	0.0785	0.0781	0.7338	0.9735	0.4233	0.1105	0.0000	6.0	0.03	
7	0.6028	1.0000	0.2677	0.4491	0.4109	0.3240	0.4666	0.7195	3.0	0.06	

Requirement 3: Prepare the data:

3. Audit missing values, duplicates, and feature types

```
In [4]: # The exported row ID is not an optical measurement and leaks source ordering
meaningful_data = raw_data.drop(columns=["Unnamed: 0"])
exact_duplicate_mask = meaningful_data.duplicated(keep="first")
clean_data = meaningful_data.loc[~exact_duplicate_mask].reset_index(drop=True)

assert clean_data.shape == (119_008, 35)

missing_by_column = clean_data.isna().sum()
missing_by_column = missing_by_column[missing_by_column > 0].sort_values(
    ascending=False
)
categorical_predictors = (
    clean_data[feature_columns]
    .select_dtypes(exclude=np.number)
    .columns.tolist()
)
repeated_feature_vectors = int(clean_data.duplicated(subset=feature_columns).sum())
nonfinite_predictor_values = int(
    (~np.isfinite(clean_data[feature_columns].to_numpy())).sum()
)

quality_summary = pd.DataFrame(
    {
        "Check": [
            "Raw rows",
            "Exported ID columns removed",
            "Exact duplicate rows removed",
            "Rows retained",
        ]
    }
)
```

```

        "Missing values retained data",
        "Non-finite clustering values",
        "Categorical clustering predictors",
        "Repeated feature vectors after exact-row deduplication",
    ],
    "Result": [
        f"{len(raw_data):,}",
        1,
        f"{int(exact_duplicate_mask.sum()):,}",
        f"{len(clean_data):,}",
        f"{int(clean_data.isna().sum().sum()):,}",
        f"{nonfinite_predictor_values:,}",
        len(categorical_predictors),
        f"{repeated_feature_vectors:,}",
    ],
}
)
quality_summary.to_csv(TABLE_DIR / "data_quality_summary.csv", index=False)
display(quality_summary)

if missing_by_column.empty:
    display(Markdown("**Missing-value result:** no missing values were found"))
else:
    display(missing_by_column.rename("Missing values").to_frame())

raw_class_counts = raw_data["Class"].astype(int).value_counts().sort_index()
clean_class_counts = clean_data["Class"].astype(int).value_counts().sort_index()
class_counts = pd.DataFrame(
    {
        "Raw rows": raw_class_counts,
        "Rows after deduplication": clean_class_counts,
        "Duplicates removed": raw_class_counts - clean_class_counts,
    }
)
class_counts.index.name = "Class"
class_counts["Clean share (%)"] = (
    100 * class_counts["Rows after deduplication"]
    / class_counts["Rows after deduplication"].sum()
)
display(class_counts.round(2))

observed_ranges = pd.DataFrame(
    {
        "Field": ["SNR", "P1-P30", "Position", "Reflectance", "loss"],
        "Minimum": [
            clean_data["SNR"].min(),
            clean_data[[f"P{i}" for i in range(1, 31)]].min().min(),
            clean_data["Position"].min(),
            clean_data["Reflectance"].min(),
            clean_data["loss"].min(),
        ],
        "Maximum": [
            clean_data["SNR"].max(),
            clean_data[[f"P{i}" for i in range(1, 31)]].max().max(),
            clean_data["Position"].max(),
            clean_data["Reflectance"].max(),
            clean_data["loss"].max(),
        ],
    }
)
display(observed_ranges)

```

	Check	Result
0	Raw rows	125,832
1	Exported ID columns removed	1
2	Exact duplicate rows removed	6,824
3	Rows retained	119,008
4	Missing values retained data	0
5	Non-finite clustering values	0
6	Categorical clustering predictors	0
7	Repeated feature vectors after exact-row dedup...	9

Missing-value result: no missing values were found.

	Raw rows	Rows after deduplication	Duplicates removed	Clean share (%)
Class				
0	16048	16044	4	13.48
1	16000	16000	0	13.44
2	16000	9180	6820	7.71
3	16000	16000	0	13.44
4	16000	16000	0	13.44
5	13849	13849	0	11.64
6	16000	16000	0	13.44
7	15935	15935	0	13.39

	Field	Minimum	Maximum
0	SNR	0.0	30.0
1	P1-P30	0.0	1.0
2	Position	0.0	0.2
3	Reflectance	-0.3	0.0
4	loss	0.0	1.0

4. Prepare clustering features

Requirement 3.1: Handle missing values (e.g., fill or drop).

No values need filling in this version of the data. A median imputer is still fitted as a reproducible no-op so the workflow safely handles future missing values.

Requirement 3.2: Convert categorical data to numeric.

The selected predictors are already numeric, so categorical encoding is not needed.

Requirement 3.3: Scale/normalize features.

P1 – P30 are already normalized within each trace, but column-wise standardization remains essential because SNR spans 0–30 while each trace coordinate spans 0–1.

```
In [5]: feature_frame = clean_data[feature_columns].copy()
assert not categorical_predictors, (
    f"Unexpected categorical predictors: {categorical_predictors}"
)
assert np.isfinite(feature_frame.to_numpy()).all()

imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()
imputed_features = imputer.fit_transform(feature_frame)
scaled_features = scaler.fit_transform(imputed_features)

preprocessing_summary = pd.DataFrame(
    {
        "Check": [
            "Rows used for K-Means",
            "Features used",
            "Missing before imputation",
            "Missing after imputation",
            "Largest |scaled mean|",
            "Largest |scaled std - 1|",
        ],
        "Result": [
            f"{scaled_features.shape[0]:,}",
            scaled_features.shape[1],
            int(feature_frame.isna().sum().sum()),
            int(np.isnan(imputed_features).sum()),
            f"{np.abs(scaled_features.mean(axis=0)).max():.2e}",
            f"{np.abs(scaled_features.std(axis=0) - 1).max():.2e}",
        ],
    }
)
display(preprocessing_summary)
```

	Check	Result
0	Rows used for K-Means	119,008
1	Features used	31
2	Missing before imputation	0
3	Missing after imputation	0
4	Largest scaled mean	2.22e-16
5	Largest scaled std - 1	1.11e-16

Results

Requirement 4: Perform clustering (e.g., K-Means), try different numbers of clusters.

5. Fit K-Means for $K = 2$ to 10

```
In [6]: evaluation_records = []
fitted_models = {}
silhouette_sample_size = min(
    analysis_config["silhouette_sample_size"], len(clean_data)
)
```

```

silhouette_indices = np.random.RandomState(random_state).permutation(
    len(clean_data)
)[:silhouette_sample_size]

for k in k_values:
    started_at = time.perf_counter()
    model = KMeans(
        n_clusters=k,
        init="k-means++",
        n_init=analysis_config["kmeans_n_init"],
        max_iter=300,
        random_state=random_state,
        algorithm="lloyd",
    )
    cluster_labels = model.fit_predict(scaled_features)
    fit_seconds = time.perf_counter() - started_at

    evaluation_records.append(
        {
            "k": k,
            "silhouette_score": silhouette_score(
                scaled_features[silhouette_indices],
                cluster_labels[silhouette_indices],
            ),
            "davies_bouldin_score": davies_bouldin_score(
                scaled_features, cluster_labels
            ),
            "inertia": model.inertia_,
            "iterations": model.n_iter_,
            "fit_seconds": fit_seconds,
        }
    )
    fitted_models[k] = model
    assert np.unique(cluster_labels[silhouette_indices]).size == k

evaluation = pd.DataFrame(evaluation_records)
evaluation.to_csv(TABLE_DIR / "kmeans_evaluation.csv", index=False)

best_silhouette_k = int(
    evaluation.loc[evaluation["silhouette_score"].idxmax(), "k"]
)
best_db_k = int(
    evaluation.loc[evaluation["davies_bouldin_score"].idxmin(), "k"]
)
best_k = best_silhouette_k
best_model = fitted_models[best_k]
assert evaluation["k"].tolist() == k_values
assert evaluation[["silhouette_score", "davies_bouldin_score"]].notna().all()

display(
    evaluation.style.format(
        {
            "silhouette_score": "{:.4f}",
            "davies_bouldin_score": "{:.4f}",
            "inertia": "{:,.0f}",
            "fit_seconds": "{:.2f}",
        }
    ).hide(axis="index")
)
display(
    Markdown(
        f"*Selected K = {best_k}* (maximum sampled Silhouette Score). "
        f"The minimum Davies-Bouldin Score occurs at *K = {best_db_k}*"
    )
)

```

```
)
)
```

k	silhouette_score	davies_bouldin_score	inertia	iterations	fit_seconds
2	0.2420	1.5394	2,693,218	8	0.71
3	0.2084	1.5017	2,298,906	12	1.29
4	0.2032	1.6926	2,036,925	9	1.22
5	0.2086	1.5493	1,810,413	10	1.40
6	0.2078	1.5215	1,702,097	20	1.40
7	0.1803	1.5046	1,622,964	33	1.73
8	0.2116	1.7739	1,542,321	14	2.13
9	0.1882	1.7179	1,479,678	19	2.95
10	0.1892	1.9050	1,441,237	17	4.47

Selected K = 2 (maximum sampled Silhouette Score). The minimum Davies–Bouldin Score occurs at **K = 3**.

Requirement 5: Evaluate with Silhouette Score or Davies–Bouldin Score.

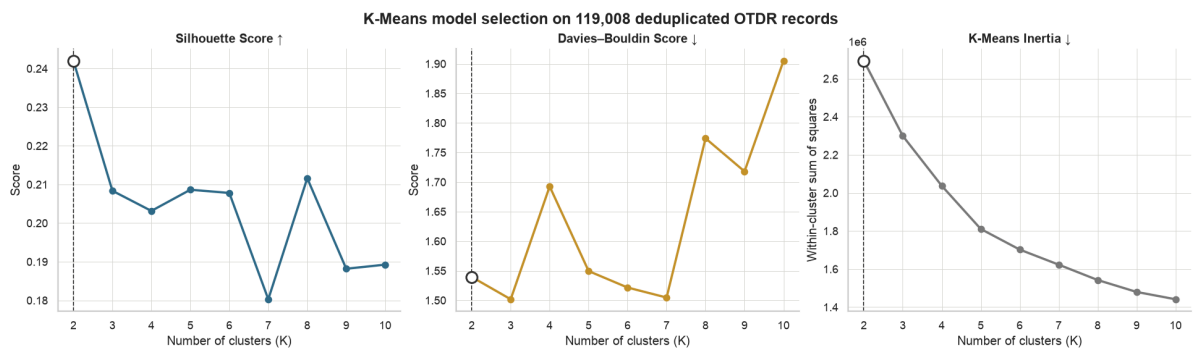
6. Compare cluster-quality metrics

```
In [7]: fig, axes = plt.subplots(1, 3, figsize=(16, 4.6), constrained_layout=True)

metric_specs = [
    ("silhouette_score", "Silhouette Score ↑", COLORS["blue"]),
    ("davies_bouldin_score", "Davies–Bouldin Score ↓", COLORS["gold"]),
    ("inertia", "K-Means Inertia ↓", COLORS["grey"]),
]
for axis, (column, title, color) in zip(axes, metric_specs, strict=True):
    axis.plot(
        evaluation["k"], evaluation[column],
        color=color, marker="o", linewidth=2.2, markersize=6,
    )
    selected_value = evaluation.loc[evaluation["k"].eq(best_k), column].iloc[0]
    axis.scatter(
        [best_k], [selected_value], s=115, facecolors="white",
        edgecolors=COLORS["charcoal"], linewidths=1.8, zorder=5,
    )
    axis.axvline(best_k, color=COLORS["charcoal"], linestyle="--", linewidth=1.8)
    axis.set(title=title, xlabel="Number of clusters (K)")
    axis.set_xticks(k_values)
    axis.spines[["top", "right"]].set_visible(False)

axes[0].set_ylabel("Score")
axes[1].set_ylabel("Score")
axes[2].set_ylabel("Within-cluster sum of squares")
axes[2].ticklabel_format(axis="y", style="sci", scilimits=(0, 0))
fig.suptitle(
    f"K-Means model selection on {len(clean_data):,} deduplicated OTDR records",
    fontsize=15,
    fontweight="bold",
)
figure_path = FIGURE_DIR / "k_selection_metrics.png"
```

```
fig.savefig(figure_path, bbox_inches="tight")
plt.show()
```



Requirement 6: Visualize the clusters and summarize your findings.

Cluster visualizations are presented in Requirement 6.1; the final synthesis is reported in the existing Takeaways section as Requirement 6.2.

Requirement 6.1: Cluster visualization

7. Visualize the selected clusters with PCA

```
In [8]: cluster_labels = best_model.labels_.astype(int)
clustered_data = clean_data.copy()
clustered_data["Cluster"] = cluster_labels

cluster_sizes = (
    pd.Series(cluster_labels, name="Cluster")
    .value_counts()
    .sort_index()
    .rename("Rows")
    .to_frame()
)

cluster_sizes["Share (%)"] = 100 * cluster_sizes["Rows"] / cluster_sizes["Rows"].sum()
assert cluster_sizes["Rows"].sum() == len(clean_data)
assert len(np.unique(cluster_labels)) == best_k
cluster_sizes.to_csv(TABLE_DIR / "cluster_sizes.csv")

pca = PCA(n_components=2, svd_solver="randomized", random_state=random_state)
pca_coordinates = pca.fit_transform(scaled_features)
centroid_coordinates = pca.transform(best_model.cluster_centers_)
assert centroid_coordinates.shape == (best_k, 2)

plot_sample_size = min(
    analysis_config["pca_plot_sample_size"], len(clean_data)
)

all_indices = np.arange(len(clean_data))
if plot_sample_size < len(clean_data):
    plot_indices, _ = train_test_split(
        all_indices,
        train_size=plot_sample_size,
        stratify=cluster_labels,
        random_state=random_state,
    )
else:
    plot_indices = all_indices
```

```

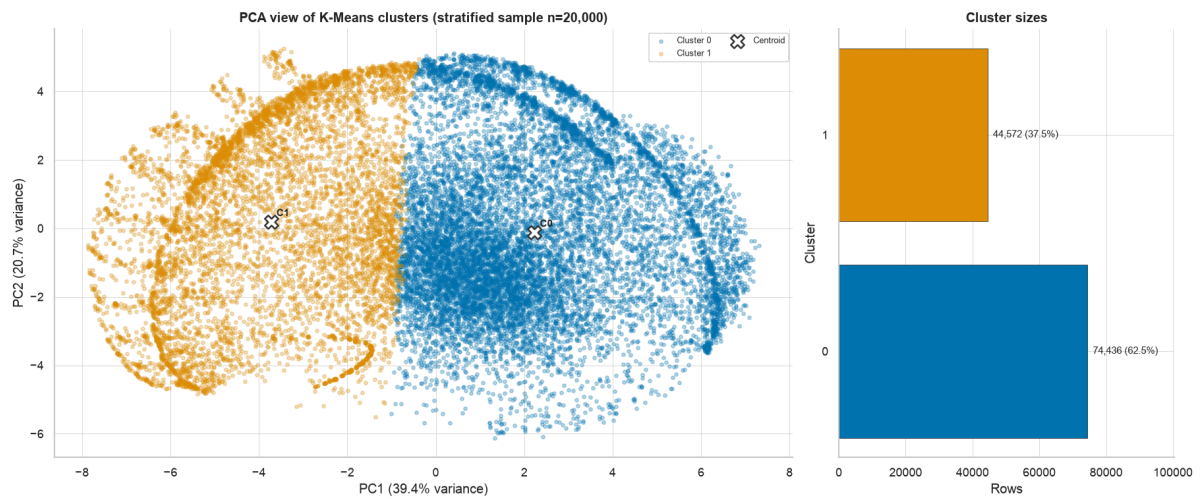
palette = sns.color_palette("colorblind", n_colors=best_k)
fig, (scatter_axis, size_axis) = plt.subplots(
    1, 2, figsize=(15, 6.2), gridspec_kw={"width_ratios": [2.2, 1]},
    constrained_layout=True,
)
for cluster_id in range(best_k):
    mask = cluster_labels[plot_indices] == cluster_id
    scatter_axis.scatter(
        pca_coordinates[plot_indices[mask], 0],
        pca_coordinates[plot_indices[mask], 1],
        s=10,
        alpha=0.28,
        color=palette[cluster_id],
        label=f"Cluster {cluster_id}",
        rasterized=True,
    )
scatter_axis.scatter(
    centroid_coordinates[:, 0], centroid_coordinates[:, 1],
    s=150, marker="X", color="white", edgecolor=COLORS["charcoal"],
    linewidth=1.5, label="Centroid", zorder=10,
)
for cluster_id, (pc1, pc2) in enumerate(centroid_coordinates):
    scatter_axis.annotate(
        f"C{cluster_id}", (pc1, pc2), xytext=(5, 5),
        textcoords="offset points", fontsize=9, fontweight="bold",
    )
scatter_axis.set(
    title=f"PCA view of K-Means clusters (stratified sample n={len(plot_indices)}",
    xlabel=f"PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)",
    ylabel=f"PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)",
)
scatter_axis.legend(loc="best", frameon=True, fontsize=8, ncol=2)
scatter_axis.spines[["top", "right"]].set_visible(False)

size_axis.barh(
    cluster_sizes.index.astype(str), cluster_sizes["Rows"],
    color=[palette[index] for index in cluster_sizes.index],
    edgecolor=COLORS["charcoal"], linewidth=0.5,
)
for cluster_id, row in cluster_sizes.iterrows():
    size_axis.text(
        row["Rows"], str(cluster_id),
        f" {int(row['Rows']):,} ({row['Share (%)']:.1f}%)",
        va="center", fontsize=9,
    )
size_axis.set(
    title="Cluster sizes", xlabel="Rows", ylabel="Cluster"
)
size_axis.spines[["top", "right"]].set_visible(False)
size_axis.set_xlim(0, cluster_sizes["Rows"].max() * 1.35)

figure_path = FIGURE_DIR / "pca_clusters_and_sizes.png"
fig.savefig(figure_path, bbox_inches="tight")
plt.show()

display(cluster_sizes.round(2))

```



Rows Share (%)

Cluster

0 74436 62.55

1 44572 37.45

8. Compare the OTDR signal profiles found by K-Means

```
In [9]: centroid_original_scale = pd.DataFrame(
    scaler.inverse_transform(best_model.cluster_centers_),
    columns=feature_columns,
    index=pd.Index(range(best_k), name="Cluster"),
)
centroid_original_scale.to_csv(TABLE_DIR / "cluster_centers_original_scale.csv")

p_columns = [f"P{index}" for index in range(1, 31)]
line_styles = ["-", "--", ":", "."]
markers = ["o", "s", "^", "D", "v", "p", "x", "<", ">", "*"]

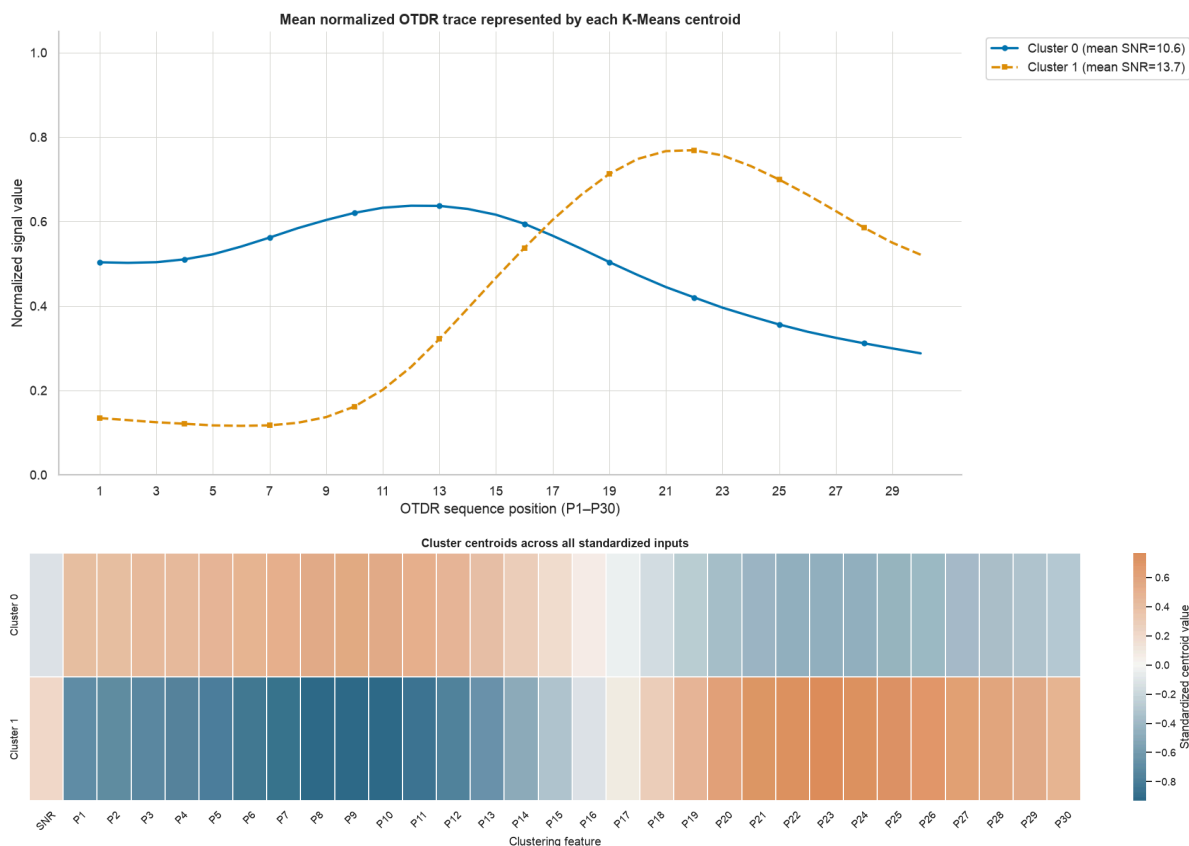
fig, axis = plt.subplots(figsize=(14, 6), constrained_layout=True)
for cluster_id in range(best_k):
    axis.plot(
        range(1, 31),
        centroid_original_scale.loc[cluster_id, p_columns],
        color=palette[cluster_id],
        linestyle=line_styles[cluster_id % len(line_styles)],
        marker=markers[cluster_id % len(markers)],
        markevery=3,
        linewidth=2,
        markersize=4,
        label=(
            f"Cluster {cluster_id} "
            f"(mean SNR={centroid_original_scale.loc[cluster_id, 'SNR']:.1f})"
        ),
    )
axis.set(
    title="Mean normalized OTDR trace represented by each K-Means centroid",
    xlabel="OTDR sequence position (P1-P30)",
    ylabel="Normalized signal value",
)
axis.set_xticks(range(1, 31, 2))
axis.set_ylim(0, 1.05)
axis.legend(bbox_to_anchor=(1.02, 1), loc="upper left", frameon=True)
axis.spines[["top", "right"]].set_visible(False)
figure_path = FIGURE_DIR / "cluster_signal_profiles.png"
```

```

fig.savefig(figure_path, bbox_inches="tight")
plt.show()

blue_orange = LinearSegmentedColormap.from_list(
    "blue_orange", ["#2F6B8A", "#F7F7F5", "#D9793D"]
)
standardized_centers = pd.DataFrame(
    best_model.cluster_centers_, columns=feature_columns,
    index=[f"Cluster {index}" for index in range(best_k)],
)
fig, axis = plt.subplots(figsize=(17, max(4.5, best_k * 0.65)), constrained=
sns.heatmap(
    standardized_centers,
    cmap=blue_orange,
    center=0,
    linewidths=0.25,
    linecolor="white",
    cbar_kws={"label": "Standardized centroid value"},
    ax=axis,
)
axis.set(
    title="Cluster centroids across all standardized inputs",
    xlabel="Clustering feature",
    ylabel="",
)
axis.tick_params(axis="x", rotation=45)
figure_path = FIGURE_DIR / "cluster_centroid_heatmap.png"
fig.savefig(figure_path, bbox_inches="tight")
plt.show()

```



9. Interpret clusters using held-out physical descriptors

Position, Reflectance, and loss were **not** used to create the clusters. Their cluster-level summaries therefore provide descriptive, post-hoc context rather than model inputs or causal explanations.

```
In [10]: interpretation_summary = (
    clustered_data.groupby("Cluster")
    .agg(
        rows=("Class", "size"),
        mean_snr=("SNR", "mean"),
        mean_position=("Position", "mean"),
        median_position=("Position", "median"),
        mean_reflectance=("Reflectance", "mean"),
        mean_loss=("loss", "mean"),
    )
    .sort_index()
)
interpretation_summary["share_pct"] = (
    100 * interpretation_summary["rows"] / interpretation_summary["rows"].sum()
)
cluster_class_counts = pd.crosstab(
    clustered_data["Cluster"], clustered_data["Class"].astype(int)
)
interpretation_summary["dominant_held_out_class"] = cluster_class_counts.idxmax(axis=1)
interpretation_summary["dominant_class_share_pct"] = (
    100 * cluster_class_counts.max(axis=1) / cluster_class_counts.sum(axis=1)
)
interpretation_summary.to_csv(TABLE_DIR / "cluster_interpretation_summary.csv")
display(interpretation_summary.round(4))
```

	rows	mean_snr	mean_position	median_position	mean_reflectance	mean_loss
Cluster						
0	74436	10.5944	0.0562	0.05	-0.0326	0.1624
1	44572	13.7124	0.0658	0.07	-0.1451	0.2792

10. Post-hoc comparison with known fault classes

This section does not turn the analysis into supervised learning. The labels are revealed only after K-Means fitting to assess whether signal-based clusters align with known event types. Cluster numbers are arbitrary and need not match class numbers.

```
In [11]: class_names = {
    0: "0 Normal",
    1: "1 Fibre tapping",
    2: "2 Bad splice",
    3: "3 Bending event",
    4: "4 Dirty connector",
    5: "5 Fibre cut",
    6: "6 PC connector",
    7: "7 Reflector",
}
known_classes = clustered_data["Class"].astype(int)
external_metrics = pd.DataFrame(
    {
        "Metric": ["Adjusted Rand Index", "Normalized Mutual Information"],
        "Value": [
            adjusted_rand_score(known_classes, cluster_labels),
            normalized_mutual_info_score(known_classes, cluster_labels),
        ],
    }
)
display(external_metrics.style.format({"Value": "{:.4f}"}).hide(axis="index"))
```

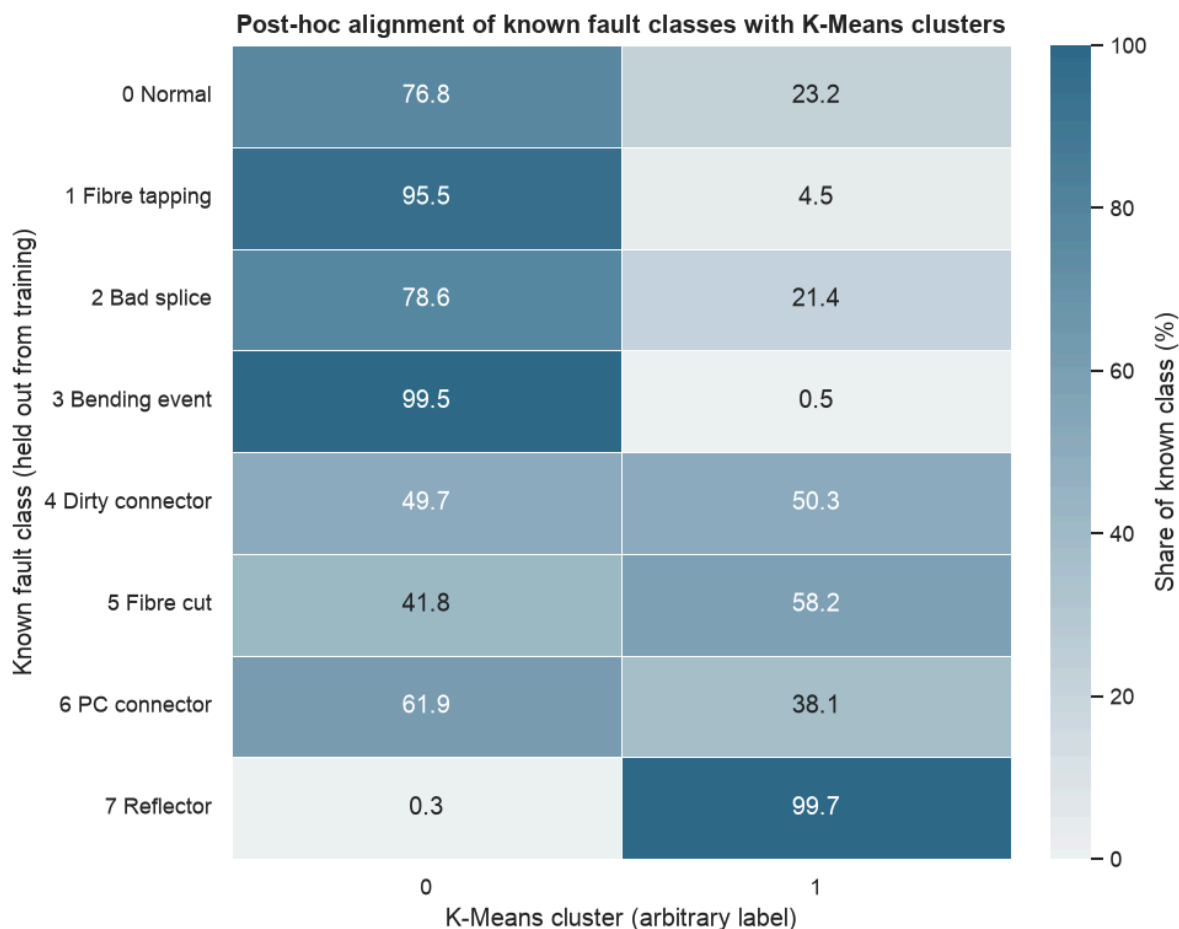
```

class_cluster_counts = pd.crosstab(
    known_classes.map(class_names), cluster_labels,
    rownames=["Known fault class"], colnames=["K-Means cluster"],
)
class_cluster_share = class_cluster_counts.div(
    class_cluster_counts.sum(axis=1), axis=0
)
assert np.allclose(class_cluster_share.sum(axis=1), 1.0)
class_cluster_counts.to_csv(TABLE_DIR / "class_cluster_counts.csv")
class_cluster_share.to_csv(TABLE_DIR / "class_cluster_row_shares.csv")

fig, axis = plt.subplots(figsize=(max(8, best_k * 0.9), 6.3), constrained_layout=True)
sns.heatmap(
    class_cluster_share * 100,
    annot=True,
    fmt=".1f",
    cmap=sns.light_palette(COLORS["blue"], as_cmap=True),
    vmin=0,
    vmax=100,
    linewidths=0.5,
    linecolor="white",
    cbar_kws={"label": "Share of known class (%)"},
    ax=axis,
)
axis.set_title("Post-hoc alignment of known fault classes with K-Means clusters")
axis.set_xlabel("K-Means cluster (arbitrary label)")
axis.set_ylabel("Known fault class (held out from training)")
axis.tick_params(axis="y", rotation=0)
figure_path = FIGURE_DIR / "class_cluster_alignment.png"
fig.savefig(figure_path, bbox_inches="tight")
plt.show()

```

Metric	Value
Adjusted Rand Index	0.0973
Normalized Mutual Information	0.1873



Takeaways

Requirement 6.2: Summary of findings

- Data preparation materially changes the evidence.** การลบ row ID และ exact duplicates ลดข้อมูลจาก 125,832 เหลือ 119,008 แถว โดยตัวอย่างซ้ำ 6,820 จาก 6,824 แถว อยู่ใน Class 2. การลบจึงป้องกันไม่ให้ OTDR traces เพียงไม่กี่แบบมีน้ำหนักซ้ำใน K-Means objective.
- K=2 is the primary internal-metric choice, with a caveat.** K=2 มี Silhouette สูงสุด (0.2420) แต่ K=3 มี Davies–Bouldin ต่ำสุด (1.5017). ค่า Silhouette ที่ค่อนข้างต่ำบ่งบอกว่ากลุ่มยัง overlap และไม่ควรรอว่าเป็น natural separation ที่สมบูรณ์.
- The centroids represent two broad waveform families.** Cluster 0 มี centroid ที่เริ่มราว 0.50 สูงขึ้นถึงช่วง P11–P13 แล้วลดลงถึง P30; Cluster 1 เริ่มต่ำราว 0.13 แล้วสูงขึ้นมากที่สุดใกล้ P21–P23. Cluster 1 ยังมี mean reflectance ตีลบมากกว่าและ mean loss สูงกว่าใน post-hoc summary.
- The clusters are not fault predictions.** เกือบทั้งหมดของ Reflector (Class 7) อยู่ใน Cluster 1 และ Bending event (Class 3) อยู่ใน Cluster 0, แต่ ARI 0.0973 และ NMI 0.1873 ยืนยันว่ากลุ่มสองกลุ่มนี้ไม่สามารถแทน fault classes แปรประเภทได้.
- Conclusion:** K-Means พบกลุ่มของ waveform shape และ SNR condition ที่กว้างกว่าชนิด fault; event position อาจทำให้รูปคลื่นเลื่อนตาม P1–P30 และเป็นส่วนหนึ่งของโครงสร้างที่โมเดลค้นพบ.

Limitations

- K-Means favors approximately spherical clusters under Euclidean distance and may split continuous signal variation rather than recover engineering fault categories.
- The PCA chart is a two-dimensional projection, not the space used for model fitting.
- The Silhouette Score is sampled for computational feasibility; Davies–Bouldin and inertia use the complete deduplicated dataset.
- Nine feature-identical pairs remain because their held-out `loss` annotations conflict; this small source inconsistency is preserved rather than resolved arbitrarily.
- The dataset license is reported by Kaggle as unknown, so the raw CSV is not embedded in this notebook or committed to the repository.

Generated artifacts

```
In [12]: generated_artifacts = sorted(
    [path.relative_to(PROJECT_ROOT) for path in FIGURE_DIR.glob("*.png")]
    + [path.relative_to(PROJECT_ROOT) for path in TABLE_DIR.glob("*.csv")]
)
display(pd.DataFrame({"Artifact": [str(path) for path in generated_artifacts]
```

	Artifact
0	outputs/figures/class_cluster_alignment.png
1	outputs/figures/cluster_centroid_heatmap.png
2	outputs/figures/cluster_signal_profiles.png
3	outputs/figures/k_selection_metrics.png
4	outputs/figures/pca_clusters_and_sizes.png
5	outputs/tables/class_cluster_counts.csv
6	outputs/tables/class_cluster_row_shares.csv
7	outputs/tables/cluster_centers_original_scale.csv
8	outputs/tables/cluster_interpretation_summary.csv
9	outputs/tables/cluster_sizes.csv
10	outputs/tables/data_quality_summary.csv
11	outputs/tables/kmeans_evaluation.csv